

 **AndreiCravtov** fix(bug): EventRouter lifetime-handling fixed, no more process crashes ()

74e9fe1 · 19 hours ago

	.githubooks	add resource monitor	10 months ago
	.github	chore(app): hardcode bug report presign...	last month
	.idea	Tweaked workspace settings (#2095)	last week
	.typings	Add node backends to model cards (#2071)	last week
	.vscode	Runner error handling (#2093)	last week
	.zed	BROKEN: work thus far	10 months ago
	app/EXO	fix(macos-app): disable URL response cac...	3 weeks ago
	bench	Integration tests infra (#1995)	2 weeks ago
	dashboard	feat: update rdma_ctl instructions (#1977)	3 weeks ago
	docs	swap camelcasemodels for frozenmodels...	last month
	nix	mlx cuda 13 (dgx spark) support (#1874)	last month
	packaging	Fix some misc build issues (#1948)	last month
	python	use custom mlx sources for linux (#2087)	last week
	resources	Add node backends to model cards (#2071)	last week
	rust	bump rust versions (#2081)	2 weeks ago
	scripts	Add download utility script (#1855)	last month
	src/exo	fix(bug): EventRouter lifetime-handling fi...	19 hours ago
	tests	Integration tests infra (#1995)	2 weeks ago
	tmp	Integration tests infra (#1995)	2 weeks ago
	tools	Integration tests infra (#1995)	2 weeks ago
	.clauderules	Add sqlite connector	10 months ago
	.cursorrules	Add RULES.md and .cursorrules	11 months ago
	.envrc	Workspace tweaks (#1849)	last month
	.gitignore	fix(bug): EventRouter lifetime-handling fi...	19 hours ago
	.python-version	Initial Structure	11 months ago
	.swift-format	fmt: add swift formatting	4 months ago
	AGENTS.md	feat: add Recent tab to model picker (#14...	3 months ago
	CLAUDE.md	docs: add AGENTS.md for AI coding agen...	4 months ago
	CONTRIBUTING.md	bump and simplify flake (#1866)	last month

 Cargo.lock	Create PID file locking for EXO (#2072)	2 weeks ago
 Cargo.toml	fmt: add taplo TOML formatter to treefmt...	3 months ago
 LICENSE	LICENSE: Fix company name/year	5 months ago
 MISSED_THINGS.md	api cancellation (#1276)	3 months ago
 PLATFORMS.md	Add Windows as a potential planned platf...	5 months ago
 README.md	bump and simplify flake (#1866)	last month
 RULES.md	Add RULES.md and .cursorrules	11 months ago
 TODO.md	cleaning up the todos (#1406)	3 months ago
 flake.lock	mlx cuda 13 (dgx spark) support (#1874)	last month
 flake.nix	use custom mlx sources for linux (#2087)	last week
 justfile	Fix some misc build issues (#1948)	last month
 package-lock.json	feat(dashboard): add Pi integration tab (#...	last month
 pyproject.toml	Add node backends to model cards (#2071)	last week
 uv.lock	use custom mlx sources for linux (#2087)	last week

 README

 Contributing

 Apache-2.0 license



exo: Run frontier AI locally. Maintained by [exo labs](#).

 Discord

 Join Server

 Follow @exolabs

 License

 Apache2.0

exo connects all your devices into an AI cluster. Not only does exo enable running models larger than would fit on a single device, but with [day-0 support for RDMA over Thunderbolt](#), makes models run faster as you add more devices.

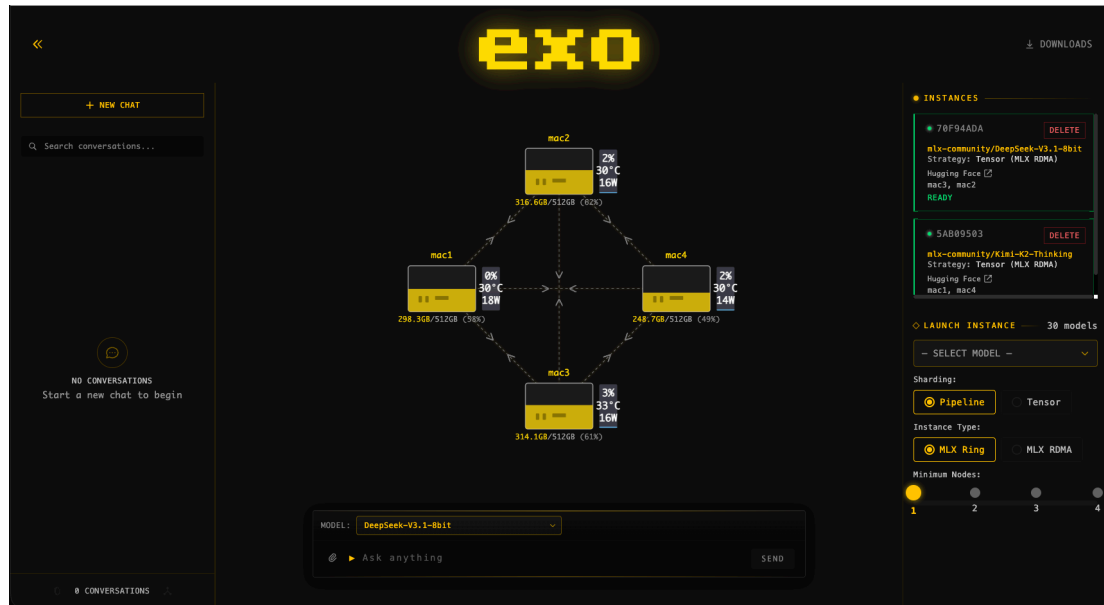
Features

- **Automatic Device Discovery:** Devices running exo automatically discover each other - no manual configuration.
- **RDMA over Thunderbolt:** exo ships with [day-0 support for RDMA over Thunderbolt 5](#), enabling 99% reduction in latency between devices.
- **Topology-Aware Auto Parallel:** exo figures out the best way to split your model across all available devices based on a realtime view of your device topology. It takes into account device resources and network latency/bandwidth between each link.
- **Tensor Parallelism:** exo supports sharding models, for up to 1.8x speedup on 2 devices and 3.2x speedup on 4 devices.

- **MLX Support:** exo uses [MLX](#) as an inference backend and [MLX distributed](#) for distributed communication.
- **Multiple API Compatibility:** Compatible with OpenAI Chat Completions API, Claude Messages API, OpenAI Responses API, and Ollama API - use your existing tools and clients.
- **Custom Model Support:** Load custom models from HuggingFace hub to expand the range of available models.

Dashboard

exo includes a built-in dashboard for managing your cluster and chatting with models.



4 × 512GB M3 Ultra Mac Studio running DeepSeek v3.1 (8-bit) and Kimi-K2-Thinking (4-bit)

Benchmarks

- ▶ Qwen3-235B (8-bit) on 4 × M3 Ultra Mac Studio with Tensor Parallel RDMA
- ▶ DeepSeek v3.1 671B (8-bit) on 4 × M3 Ultra Mac Studio with Tensor Parallel RDMA
- ▶ Kimi K2 Thinking (native 4-bit) on 4 × M3 Ultra Mac Studio with Tensor Parallel RDMA

Quick Start

Devices running exo automatically discover each other, without needing any manual configuration. Each device provides an API and a dashboard for interacting with your cluster (runs at `http://localhost:52415`).

There are two ways to run exo:

Run from Source (macOS)

If you have [Nix](#) installed, you can skip most of the steps below and run exo directly:

```
nix run .#exo
```

Note: To accept the Cachix binary cache (and avoid the Xcode Metal ToolChain), add to `/etc/nix/nix.conf` :

```
trusted-users = root      (or your username)
experimental-features = nix-command flakes
```

Then restart the Nix daemon: `sudo launchctl kickstart -k system/org.nixos.nix-daemon`

Prerequisites:

- [Xcode](#) (provides the Metal ToolChain required for MLX compilation)
- [brew](#) (for simple package management on macOS)

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

- [uv](#) (for Python dependency management)
- [node](#) (for building the dashboard)

```
brew install uv node
```

- [rust](#) (to build Rust bindings, nightly for now)

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustup toolchain install nightly
```

- [macmon](#) (for hardware monitoring on Apple Silicon)

Install the pinned fork revision used by this repo instead of Homebrew `macmon`. Homebrew `macmon 0.6.1` still crashes on Apple M5.

```
cargo install --git https://github.com/vladkens/macmon \
  --rev a1cd06b6cc0d5e61db24fd8832e74cd992097a7d \
  macmon \
  --force
```

Clone the repo, build the dashboard, and run exo:

```
# Clone exo
git clone https://github.com/exo-explore/exo

# Build dashboard
cd exo/dashboard && npm install && npm run build && cd ..

# Run exo
uv run exo
```

This starts the exo dashboard and API at <http://localhost:52415/>

Please view the section on RDMA to enable this feature on MacOS >=26.2!

Run from Source (Linux)**Prerequisites:**

- [uv](#) (for Python dependency management)
- [node](#) (for building the dashboard) - version 18 or higher
- [rust](#) (to build Rust bindings, nightly for now)

Installation methods:**Option 1: Using system package manager (Ubuntu/Debian example):**

```
# Install Node.js and npm
sudo apt update
sudo apt install nodejs npm

# Install uv
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# Install Rust (using rustup)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustup toolchain install nightly
```

Option 2: Using Homebrew on Linux (if preferred):

```
# Install Homebrew on Linux
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

# Install dependencies
brew install uv node

# Install Rust (using rustup)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustup toolchain install nightly
```

Note: The `macmon` package is macOS-only and not required for Linux.

Clone the repo, build the dashboard, and run `exo`:

```
# Clone exo
git clone https://github.com/exo-explore/exo

# Build dashboard
cd exo/dashboard && npm install && npm run build && cd ..

# Run exo
uv run exo
```

This starts the `exo` dashboard and API at <http://localhost:52415/>

Important note for Linux users: Currently, `exo` runs on CPU on Linux. GPU support for Linux platforms is under development. If you'd like to see support for your specific Linux hardware, please [search for existing feature requests](#) or create a new one.

Configuration Options:

- `--no-worker` : Run `exo` without the worker component. Useful for coordinator-only nodes that handle networking and orchestration but don't execute inference tasks. This is helpful for machines without sufficient GPU resources but with good network connectivity.

```
uv run exo --no-worker
```

File Locations (Linux):

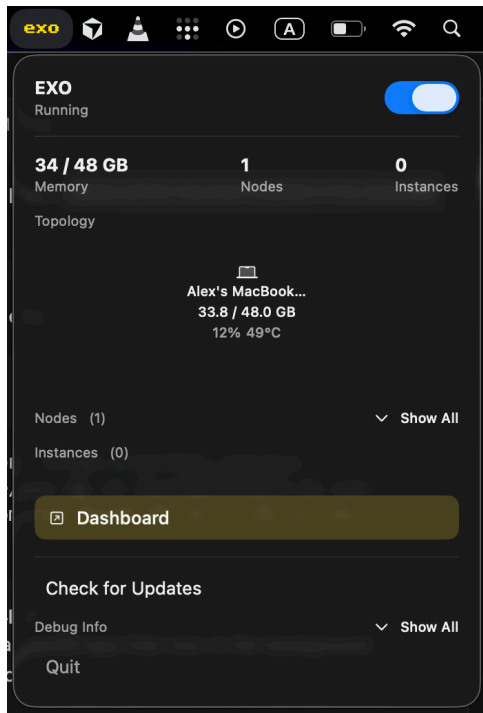
`exo` follows the [XDG Base Directory Specification](#) on Linux:

- **Configuration files:** `~/.config/exo/` (or `$XDG_CONFIG_HOME/exo/`)
- **Data files:** `~/.local/share/exo/` (or `$XDG_DATA_HOME/exo/`)
- **Cache files:** `~/.cache/exo/` (or `$XDG_CACHE_HOME/exo/`)
- **Log files:** `~/.cache/exo/exo_log/` (with automatic log rotation)
- **Custom model cards:** `~/.local/share/exo/custom_model_cards/`

You can override these locations by setting the corresponding XDG environment variables.

macOS App

`exo` ships a macOS app that runs in the background on your Mac.



The macOS app requires macOS Tahoe 26.2 or later.

Download the latest build here: [EXO-latest.dmg](#).

The app will ask for permission to modify system settings and install a new Network profile. Improvements to this are being worked on.

Custom Namespace for Cluster Isolation:

The macOS app includes a custom namespace feature that allows you to isolate your exo cluster from others on the same network. This is configured through the `EXO_LIBP2P_NAMESPACE` setting:

- **Use cases:**
 - Running multiple separate exo clusters on the same network
 - Isolating development/testing clusters from production clusters
 - Preventing accidental cluster joining
- **Configuration:** Access this setting in the app's Advanced settings (or set the `EXO_LIBP2P_NAMESPACE` environment variable when running from source)

The namespace is logged on startup for debugging purposes.

Uninstalling the macOS App

The recommended way to uninstall is through the app itself: click the menu bar icon → Advanced → Uninstall. This cleanly removes all system components.

If you've already deleted the app, you can run the standalone uninstaller script:

```
sudo ./app/EXO/uninstall-exo.sh
```



This removes:

- Network setup LaunchDaemon
- Network configuration script
- Log files
- The "exo" network location

Note: You'll need to manually remove EXO from Login Items in System Settings → General → Login Items.

Enabling RDMA on macOS

RDMA is a new capability added to macOS 26.2. It works on any Mac with Thunderbolt 5 (M4 Pro Mac Mini, M4 Max Mac Studio, M4 Max MacBook Pro, M3 Ultra Mac Studio).

Please refer to the caveats for immediate troubleshooting.

To enable RDMA on macOS, follow these steps:

1. Shut down your Mac.
2. Hold down the power button for 10 seconds until the boot menu appears.
3. Select "Options" to enter Recovery mode.
4. When the Recovery UI appears, open the Terminal from the Utilities menu.
5. In the Terminal, type:

```
rdma_ctl enable
```



and press Enter.

6. Reboot your Mac.

After that, RDMA will be enabled in macOS and exo will take care of the rest.

Important Caveats

1. Devices that wish to be part of an RDMA cluster must be connected to all other devices in the cluster.
2. The cables must support TB5.
3. On a Mac Studio, you cannot use the Thunderbolt 5 port next to the Ethernet port.
4. If running from source, please use the script found at `tmp/set_rdma_network_config.sh` , which will disable Thunderbolt Bridge and set dhcp on each RDMA port.
5. RDMA ports may be unable to discover each other on different versions of MacOS. Please ensure that OS versions match exactly (even beta version numbers) on all devices.

Environment Variables

exo supports several environment variables for configuration:

Variable	Description	Default
EXO_DEFAULT_MODELS_DIR	Default directory for model downloads and caches. Always first in the writable dirs list.	<code>~/.local/share/exo/models</code> (Linux) or <code>~/.exo/models</code> (macOS)
EXO_MODELS_DIRS	Colon-separated additional writable directories for model downloads. Checked in order after the default; first with enough free space is used.	None
EXO_MODELS_READ_ONLY_DIRS	Colon-separated read-only directories to search for pre-downloaded models (e.g., NFS mounts, shared storage). Models here cannot be deleted.	None
EXO_OFFLINE	Run without internet connection (uses only local models)	false
EXO_ENABLE_IMAGE_MODELS	Enable image model support	false
EXO_LIBP2P_NAMESPACE	Custom namespace for cluster isolation	None
EXO_FAST_SYNC	Control MLX_METAL_FAST_SYNC behavior (for JACCL backend)	Auto
EXO_TRACING_ENABLED	Enable distributed tracing for performance analysis	false

Example usage:



```
# Use pre-downloaded models from NFS mount (read-only)
EXO_MODELS_READ_ONLY_DIRS=/mnt/nfs/models:/opt/ai-models uv run exo

# Download models to an external SSD (falls back to default dir if full)
EXO_MODELS_DIRS=/Volumes/ExternalSSD/exo-models uv run exo

# Run in offline mode
EXO_OFFLINE=true uv run exo

# Enable image models
EXO_ENABLE_IMAGE_MODELS=true uv run exo

# Use custom namespace for cluster isolation
EXO_LIBP2P_NAMESPACE=my-dev-cluster uv run exo
```

Using the API

exo provides multiple API-compatible interfaces for maximum compatibility with existing tools:

- **OpenAI Chat Completions API** - Compatible with OpenAI clients
- **Claude Messages API** - Compatible with Anthropic's Claude format
- **OpenAI Responses API** - Compatible with OpenAI's Responses format
- **Ollama API** - Compatible with Ollama and tools like OpenWebUI

If you prefer to interact with exo via the API, here is an example creating an instance of a small model (`mlx-community/Llama-3.2-1B-Instruct-4bit`), sending a chat completions request and deleting the instance.

1. Preview instance placements

The `/instance/previews` endpoint will preview all valid placements for your model.

```
curl "http://localhost:52415/instance/previews?model_id=llama-3.2-1b"
```



Sample response:

```
{
  "previews": [
    {
      "model_id": "mlx-community/Llama-3.2-1B-Instruct-4bit",
      "sharding": "Pipeline",
      "instance_meta": "MlxRing",
      "instance": {...},
      "memory_delta_by_node": {"local": 729808896},
      "error": null
    }
    // ...possibly more placements...
  ]
}
```



This will return all valid placements for this model. Pick a placement that you like. To pick the first one, pipe into `jq` :

```
curl "http://localhost:52415/instance/previews?model_id=llama-3.2-1b" | jq -c '.previews[] | select(.error == null) |'
```



2. Create a model instance

Send a POST to `/instance` with your desired placement in the `instance` field (the full payload must match types as in `CreateInstanceParams`), which you can copy from step 1:

```
curl -X POST http://localhost:52415/instance \
-H 'Content-Type: application/json' \
-d '{
  "instance": {...}
}'
```

Sample response:

```
{
  "message": "Command received.",
  "command_id": "e9d1a8ab-..."
}
```

3. Send a chat completion

Now, make a POST to `/v1/chat/completions` (the same format as OpenAI's API):

```
curl -N -X POST http://localhost:52415/v1/chat/completions \
-H 'Content-Type: application/json' \
-d '{
  "model": "mlx-community/Llama-3.2-1B-Instruct-4bit",
  "messages": [
    {"role": "user", "content": "What is Llama 3.2 1B?"}
  ],
  "stream": true
}'
```

4. Delete the instance

When you're done, delete the instance by its ID (find it via `/state` or `/instance` endpoints):

```
curl -X DELETE http://localhost:52415/instance/YOUR_INSTANCE_ID
```

Claude Messages API Compatibility

Use the Claude Messages API format with the `/v1/messages` endpoint:

```
curl -N -X POST http://localhost:52415/v1/messages \
-H 'Content-Type: application/json' \
-d '{
  "model": "mlx-community/Llama-3.2-1B-Instruct-4bit",
  "messages": [
    {"role": "user", "content": "Hello"}
  ],
  "max_tokens": 1024,
  "stream": true
}'
```

OpenAI Responses API Compatibility

Use the OpenAI Responses API format with the `/v1/responses` endpoint:

```
curl -N -X POST http://localhost:52415/v1/responses \
-H 'Content-Type: application/json' \
-d '{
  "model": "mlx-community/Llama-3.2-1B-Instruct-4bit",
  "messages": [
    {"role": "user", "content": "Hello"}
  ],
}
```

```
"stream": true
}'
```

Ollama API Compatibility

exo supports Ollama API endpoints for compatibility with tools like OpenWebUI:

```
# Ollama chat
curl -X POST http://localhost:52415/ollama/api/chat \
-H 'Content-Type: application/json' \
-d '{
  "model": "mlx-community/Llama-3.2-1B-Instruct-4bit",
  "messages": [
    {"role": "user", "content": "Hello"}
  ],
  "stream": false
}'

# List models (Ollama format)
curl http://localhost:52415/ollama/api/tags
```



Custom Model Loading from HuggingFace

You can add custom models from the HuggingFace hub:

```
curl -X POST http://localhost:52415/models/add \
-H 'Content-Type: application/json' \
-d '{
  "model_id": "mlx-community/my-custom-model"
}'
```



Security Note:

Custom models requiring `trust_remote_code` in their configuration must be explicitly enabled (default is false) for security. Only enable this if you trust the model's remote code execution. Models are fetched from HuggingFace and stored locally as custom model cards.

*Other useful API endpoints:**

- List all models: `curl http://localhost:52415/models`
- List downloaded models only: `curl http://localhost:52415/models?status=downloaded`
- Search HuggingFace: `curl "http://localhost:52415/models/search?query=llama&limit=10"`
- Inspect instance IDs and deployment state: `curl http://localhost:52415/state`

For further details, see:

- API documentation in [docs/api.md](#).
- API types and endpoints in [src/exo/master/api.py](#).

Benchmarking

The `exo-bench` tool measures model prefill and token generation speed across different placement configurations. This helps you optimize model performance and validate improvements.

Prerequisites:

- Nodes should be running with `uv run exo` before benchmarking
- The tool uses the `/bench/chat/completions` endpoint

Basic usage:

```
uv run bench/exo_bench.py \
  --model Llama-3.2-1B-Instruct-4bit \
  --pp 128,256,512 \
  --tg 128,256
```

Key parameters:

- --model : Model to benchmark (short ID or HuggingFace ID)
- --pp : Prompt size hints (comma-separated integers)
- --tg : Generation lengths (comma-separated integers)
- --max-nodes : Limit placements to N nodes (default: 4)
- --instance-meta : Filter by ring , jaccl , or both (default: both)
- --sharding : Filter by pipeline , tensor , or both (default: both)
- --repeat : Number of repetitions per configuration (default: 1)
- --warmup : Warmup runs per placement (default: 0)
- --json-out : Output file for results (default: bench/results.json)

Example with filters:

```
uv run bench/exo_bench.py \
  --model Llama-3.2-1B-Instruct-4bit \
  --pp 128,512 \
  --tg 128 \
  --max-nodes 2 \
  --sharding tensor \
  --repeat 3 \
  --json-out my-results.json
```

The tool outputs performance metrics including prompt tokens per second (prompt_tps), generation tokens per second (generation_tps), and peak memory usage for each configuration.

Hardware Accelerator Support

Releases 16

 **v1.0.71** Latest

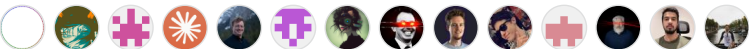
last month

[+ 15 releases](#)

Packages

No packages published

Contributors 100



[+ 86 contributors](#)

Languages

